

Laporan Praktikum P6 - Polymorphism, Method Overriding, Method Overloading, dan Dynamic Binding

Nama : Akhmad Satrio Cahyo Pratama | NRP : 3125522013 | Studi : Teknik Informatika

1. Sprint Goal

Mengembangkan behavior object pada project P5 melalui method overriding dan polymorphism, sehingga subclass merespons pemanggilan method yang sama dengan perilaku berbeda. P6 adalah refactoring dari P5, bukan project baru.

2. Sprint Backlog

ID	Sprint Backlog	Status
SB-01	Audit superclass dan subclass P5	DONE
SB-02	Menentukan behavior yang dapat di-override	DONE
SB-03	Implementasi overriding pada subclass	DONE
SB-04	Implementasi overloading yang relevan	DONE
SB-05	Implementasi upcasting	DONE
SB-06	Membuat polymorphic collection	DONE
SB-07	Melakukan pengujian dynamic binding	DONE

3. Hierarchy Class P5

P5 menyediakan superclass **Entitas** dengan subclass **Nasabah** dan **BankSampah**. Karena hanya ada dua subclass, P6 menambah subclass ketiga **Koperasi** agar polymorphic collection memiliki tiga tipe object.

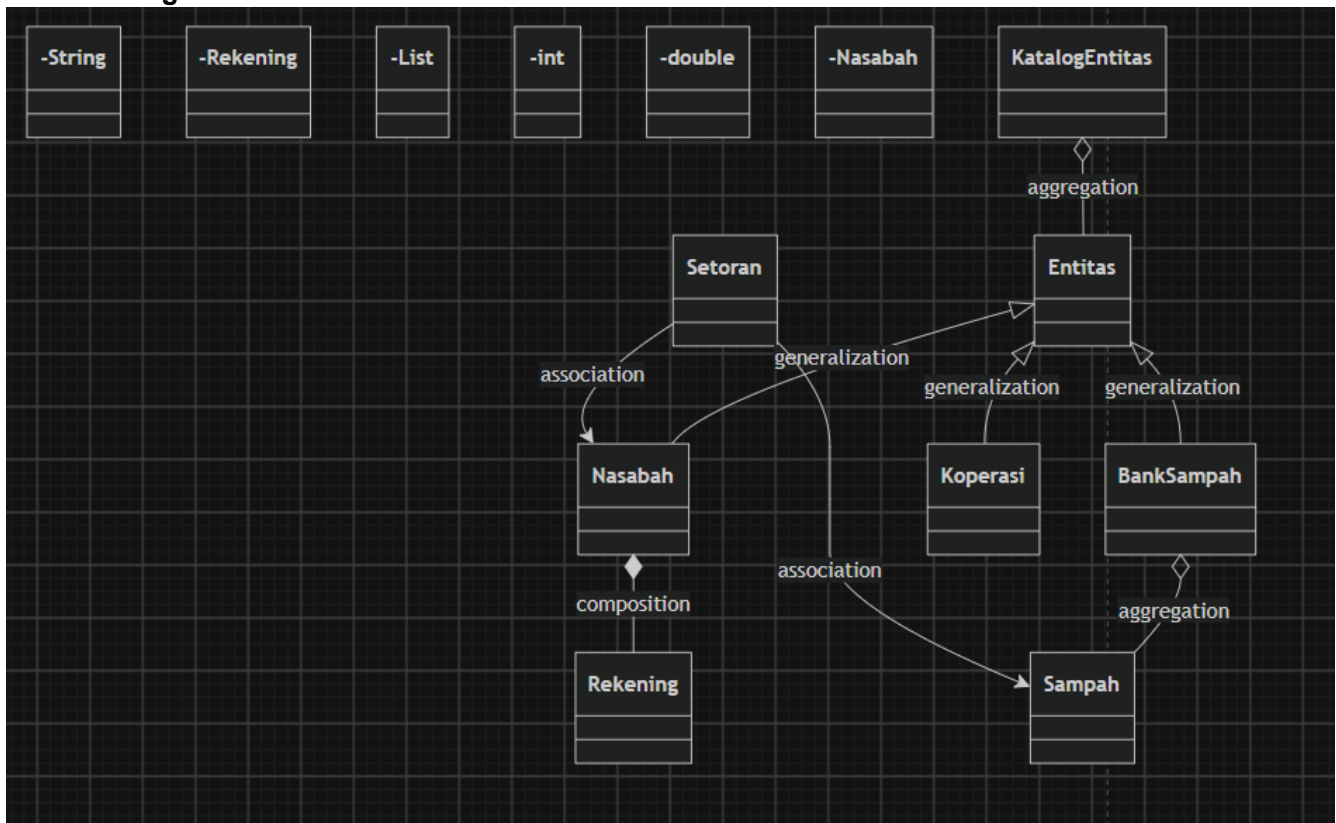
```
+-----+
| Entitas (superclass) | - nama
| +getPeran()          | - alamat
| +tampilkanPeran()    | +getNama() getAlamat()
| +hitungKontribusi()  | +getPeran() tampilkanPeran()
+-----+             +hitungKontribusi()
      ^               ^
      |               | generalization
+-----+ +-----+ +-----+
| Nasabah | | BankSa | | Koperasi |
| +id     | | +izin  | | +kode     |
| +rekening | | +smpah | | +anggota |
+-----+ +-----+ | +modal  |
                        +-----+
```

4. Behavior yang Dipilih untuk Polymorphism

Audit menemukan tiga behavior milik superclass yang memiliki implementasi berbeda pada setiap subclass. Ketiganya dijadikan behavior polymorphic.

Superclass	Subclass	Method	Implementasi pada Subclass
Entitas	Nasabah	getPeran()	"Nasabah"
Entitas	Nasabah	tampilkanPeran()	identitas + id nasabah + no rekening
Entitas	Nasabah	hitungKontribusi()	saldo rekening nasabah
Entitas	BankSampah	getPeran()	"Bank Sampah"
Entitas	BankSampah	tampilkanPeran()	identitas + nomor izin + jumlah sampah
Entitas	BankSampah	hitungKontribusi()	total nilai sampah ditampung
Entitas	Koperasi	getPeran()	"Koperasi"
Entitas	Koperasi	tampilkanPeran()	identitas + kode + modal per anggota
Entitas	Koperasi	hitungKontribusi()	total modal koperasi

5. Class Diagram Terbaru



Relasi P4 yang dipertahankan: **association** (Setoran memakai Nasabah dan Sampah), **aggregation** (BankSampah memiliki Sampah), **composition** (Nasabah memiliki bagian Rekening). Relasi P5: **generalization** antar Entitas dengan ketiga subclass.

6. Contoh Method Overriding

```

public class Entitas {
    // superclass
    public String getPeran() { return "Entitas"; } public
    void tampilkanPeran() { System.out.println("Peran
    : " + getPeran());
    System.out.println("Kontrib : Rp" + hitungKontribusi());
    public double hitungKontribusi() { return 0; } } public
    class Nasabah extends Entitas { // subclass @Override
    public String getPeran() { return "Nasabah"; } @Override
    public void tampilkanPeran() {
        super.tampilkanPeran();
        // bagian generik dari superclass
        System.out.println("ID : " + id + " / Rek : " + rekening.getNomorRekening());
    }
    @Override public double hitungKontribusi() { return rekening.getSaldo(); }
    } public class Koperasi extends Entitas { // subclass @Override public
    String getPeran() { return "Koperasi"; } @Override public void
    tampilkanPeran() {
        super.tampilkanPeran();
        System.out.println("Modal/Anggota : Rp" + hitungModalPerAnggota());
    }
    @Override public double hitungKontribusi() { return modal; }
    }
}
    
```

7. Contoh Method Overloading

Overloading dipakai saat satu nama method melayani kebutuhan berbeda. Alasan penggunaan dinyatakan eksplisit pada tiap class.

Class	Overload	Alasan penggunaan
BankSampah	cariSampah(kode) -> Sampah	kode unik, hasil satu object
BankSampah	cariSampah(jenis, minHarga) -> List	jenis + ambang harga, hasil banyak data
BankSampah	cariSampah(maksHarga) -> List	saring sampah di bawah batas harga
Rekening	tambahSaldo(nominal)	setoran biasa, cukup menambah saldo
Rekening	tambahSaldo(nominal, keterangan)	sumber setoran perlu dicatat di laporan
Koperasi	tambahAnggota()	pendaftaran perorangan
Koperasi	tambahAnggota(jumlah)	pendaftaran massal sekaligus
KatalogEntitas	cari(nama) -> Entitas	pencarian satu object berdasarkan nama
KatalogEntitas	cari(minKontribusi) -> List	saring entitas dengan nilai minimal

KatalogEntitas	cari(peran, minKontribusi) -> List	kombinasi peran dan ambang nilai
----------------	------------------------------------	----------------------------------

8. Penjelasan Upcasting

Upcasting adalah memperlakukan object subclass sebagai superclass, baik secara eksplisit lewat assignment maupun langsung pada deklarasi variable. Object asli tidak berubah dan tidak kehilangan data; yang berubah hanya cara aksesnya. Setelah upcasting, reference hanya bisa memanggil member milik superclass, tetapi method yang dieksekusi tetap ditentukan tipe aktual object.

```
Nasabah nas = new Nasabah("NS-001", "Budi", "Jl. Raya Sumenep 10", 10000);
```

```
Entitas upcast = nas; // upcasting eksplisit
Entitas e1 = new Nasabah(...); // upcasting langsung
Entitas e2 = new BankSampah(...);
Entitas e3 = new Koperasi(...); Entitas[] daftar = { nas1, nas2, nas3, bank, kop
}; // polymorphic reference for (Entitas e : daftar) { e.tampilkanPeran();
total += e.hitungKontribusi(); }
```

9. Screenshot Hasil Running

```
$ javac -encoding UTF-8 *.java
$ java Main

=== SISTEM MANAJEMEN BANK SAMPAH ===
P6 - Polymorphism, Method Overriding, Method Overloading, dan Dynamic Binding

===== BAGIAN B - METHOD OVERRIDING ===== [P6.1]
Object bertipe Nasabah memanggil method milik Nasabah:
Peran      : Nasabah
Nama       : Budi
Nilai Kontribusi : Rp1.000
ID Nasabah : NS-001
No Rekening : R-NS-001

[P6.2] Object bertipe BankSampah memanggil method milik BankSampah:
Peran      : Bank Sampah
Nilai Kontribusi : Rp19.500
Nomor Izin : IZN-2026-001
Jumlah Sampah : 3 jenis (19.500 poin/kg)

[P6.3] Object bertipe Koperasi memanggil method milik Koperasi:
Peran      : Koperasi
Nilai Kontribusi : Rp75.000.000
Kode Koperasi : KOP-01
Jumlah Anggota : 45
Modal per Anggota: Rp1.666.667

===== BAGIAN C - METHOD OVERLOADING ===== cariSampah("SP-002")
-> Koran Bekas cariSampah("Logam", 10000.0) -> 1 data: SP-003 Kaleng Bekas
cariSampah(5000.0) -> 2 data: SP-001 Botol, SP-002 Koran
tambahAnggota() -> 46 anggota tambahAnggota(3) ->
49 anggota cari("Budi") -> Budi (Nasabah) cari(10000.0)
-> 3 entitas: Budi, Bank Sampah PENS, Koperasi

===== BAGIAN E - POLYMORPHIC COLLECTION =====
tipe array : Entitas[] jumlah : 5 total kontribusi
seluruh entitas : Rp75.047.000 jumlah entitas
(List<Entitas>) : 5
```

10. Tabel Pengujian Dynamic Binding

No	Tipe Reference	Tipe Object	Method	Output	Nilai Kontribusi
1	Entitas	Nasabah	tampilkanPeran()	Nasabah	Rp20.000
2	Entitas	BankSampah	tampilkanPeran()	Bank Sampah	Rp19.500
3	Entitas	Koperasi	tampilkanPeran()	Koperasi	Rp75.000.000

Mengapa method yang dijalankan berbeda walaupun reference bertipe sama? Reference hanya menyimpan tipe statis (Entitas) dan alamat object. Saat runtime JVM membaca tipe aktual object pada object header, yaitu Nasabah, BankSampah, atau Koperasi, lalu mencari method yang cocok pada tipe aktual tersebut, bukan pada tipe reference. Karena setiap subclass meng-override method, implementasi yang dipanggil adalah milik subclass. Proses ini disebut **Dynamic Binding** atau **Dynamic Method Dispatch** dan terjadi saat runtime.

11. Sprint Review

Item	Hasil
Superclass dan subclass tersedia	Entitas + Nasabah, BankSampah, Koperasi (3 subclass)
Method overriding berhasil	3 method x 3 subclass, semua memakai @Override
Min 2 subclass punya behavior berbeda	3 subclass: Nasabah, BankSampah, Koperasi
Method overloading tersedia	4 class: BankSampah, Rekening, Koperasi, KatalogEntitas
Upcasting berhasil	3 upcasting eksplisit + polymorphic reference
Polymorphic collection berhasil	array 5 object dan List 5 object, 3 tipe
Dynamic binding terbukti	Tabel bagian 10: 3 reference Entitas, 3 hasil berbeda
Program berjalan	javac dan java berhasil, tanpa error
Kendala	Awalnya hanya 2 subclass sehingga collection kurang variatif, ditambah subclass Koperasi

12. Sprint Retrospective

What Went Well? Behavior polymorphic berhasil pada tiga method `getPeran()`, `tampilkanPeran()`, dan `hitungKontribusi()`. Satu loop pada polymorphic collection menampilkan tiga format berbeda tanpa percabangan if-else, dan relasi P4 tetap utuh setelah refactoring.

What Went Wrong? Sulit membedakan overloading dan overriding. Overriding menyamar saat nama method sama dan dipanggil lewat reference superclass, sedangkan overloading selalu terlihat karena parameter berbeda dan dipilih pada compile time.

Improvement. Entitas masih punya implementasi generik yang dapat dipanggil walaupun Entitas tidak pernah diinstansikan langsung. Pada P7 class ini sebaiknya menjadi abstract class dengan abstract method agar tiap subclass wajib mengimplementasikan behavior-nya sendiri.